

实时协作文档编辑系统实现日程

大帅，基于你提供的项目描述（Spring Boot 后端 + WebSocket 实时编辑 + MongoDB 存储 + JWT 认证 + Redis 缓存），我为你制定了一个具体的实现日程。假设你的寒假从2026年2月2日（周一）开始（当前是2月1日晚，留一天缓冲），总时长4周（28天），每天开发时间建议4-6小时（避免烧脑，留时间调试/休息）。日程按周分段，每周有里程碑，每日有具体任务，体现渐进式开发：先基础后核心后优化。

- 总体原则：先跑通 MVP（基本编辑 + 实时广播），再加冲突解决 + 扩展。每天结束 commit & push（用 GitHub 仓库 real-time-collaborative-rj）。
- 资源准备：MongoDB Atlas 已配好；Redis 用 Docker 跑（`docker run -d -p 6379:6379 redis`）；前端用 React + Quill.js + @stomp/stompjs（简单 CRA 初始化）。
- 测试方式：Postman 测试 API；多浏览器窗口模拟并发。
- 风险缓冲：每周留1天“调试日”，如果卡住（如 WebSocket 断连），可以问我调试代码。

周1：基础搭建 + 用户认证（2月2日-2月8日，目标：能登录 + 创建文档）

- 日1 (2/2, 周一)：项目初始化 + 依赖配置
 - 用 Spring Initializr 生成项目（依赖：Web, Security, WebSocket, Data MongoDB, Data Redis, Lombok, JWT）。
 - 配置 application.yml：MongoDB URI + Redis host/port + JWT secret。
 - 写 Hello Controller 测试跑通（GET /hello）。
 - commit: "init: 项目基础结构 + 配置"
- 日2 (2/3, 周二)：用户实体 + Repository + Service
 - 定义 User 实体（@Document, id, username, password - BCrypt）。
 - UserRepository extends MongoRepository<User, String>。
 - UserService：register (加密密码 + save), findByUsername。
 - commit: "feat: 用户模块基础"

- 日3 (2/4, 周三): JWT 认证配置

- 写 JwtTokenProvider (generateToken, validate, getUserId)。
- SecurityConfig: disable CSRF, stateless, add JwtFilter。
- AuthController: POST /auth/register, POST /auth/login (返回 JWT)。
- 测试: Postman 注册/登录, 获取 token。
- commit: "feat: JWT 认证实现"

- 日4 (2/5, 周四): 文档实体 + CRUD 基础

- 定义 DocumentEntity (@Document, id, title, content, ownerId, version - @Version, versions List)。
- DocumentRepository extends MongoRepository<DocumentEntity, String>。
- DocumentService: create (set owner), getById, listByUser。
- commit: "feat: 文档实体 + Repository"

- 日5 (2/6, 周五): 文档 Controller

- DocumentController: POST /documents (创建), GET /documents (列表), GET /documents/{id} (需 @Authenticated)。
- 加权限: owner 才能编辑。
- 测试: 用 token 调用 API 创建/获取文档。
- commit: "feat: 文档 CRUD API"

- 日6 (2/7, 周六): 调试日 + 前端基础

- 修复周内 bug (如 Mongo 连不上、JWT 失效)。
- 初始化 React 前端: CRA, 加 Axios, 登录页 + 文档列表页。

- commit: "fix: 周1 bug 修复 + 前端 stub"
- 日7 (2/8, 周日): 休息/复习 (看 Spring Security/WebSocket 文档)。
- 里程碑: 后端跑通认证 + 文档 CRUD, 前端能登录看到空文档列表。

周2: WebSocket 集成 + 基础实时广播 (2月9日-2月15日, 目标: 多人看到编辑变化)

- 日8 (2/9, 周一): WebSocket 配置
 - 新建 WebSocketConfig implements WebSocketMessageBrokerConfigurer。
 - 启用 STOMP: registerStompEndpoints("/ws"), configureMessageBroker (enableSimpleBroker("/topic"))。
 - 加 WebSocket 认证: ChannelInterceptor 解析 CONNECT 帧的 JWT。
 - commit: "feat: WebSocket STOMP 配置"
- 日9 (2/10, 周二): 实时编辑广播
 - DocumentController 加 WebSocket 端点:
@Messaging("/edit/{docId}") → @SendTo("/topic/{docId}") (广播 String delta)。
 - 更新 DocumentService: applyDelta (简单合并 content)。
 - commit: "feat: 基础广播实现"
- 日10 (2/11, 周三): 前端集成 WebSocket
 - 前端用 @stomp/stompjs 连接 /ws (带 JWT header)。
 - Quill.js 编辑器: onChange 发送 delta 到 /app/edit/{docId}。
 - 订阅 /topic/{docId} 应用接收 delta。
 - 测试: 两人打开文档, A 打字 B 看到 (可能覆盖)。
 - commit: "feat: 前端 WebSocket + Quill 集成"
- 日11 (2/12, 周四): Redis 集成会话缓存
 - 配置 Spring Data Redis: RedisTemplate。

- OnlineUserService: 用 Redis Set 存 "online:{docId}" → userIds。
- WebSocket 连接时 add 用户, 断开 remove。
- commit: "feat: Redis 缓存在线用户"

- 日12 (2/13, 周五): 前端显示在线用户
 - 前端订阅在线状态, 显示 “X 人正在编辑”。
 - 加心跳: 每 30s 发送 ping 保持连接。
 - commit: "feat: 在线状态显示"

- 日13 (2/14, 周六): 调试日
 - 修复 WebSocket 断连/重连 bug (stomp.js auto reconnect)。
 - 测试并发: 3 人编辑看广播是否乱。
 - commit: "fix: 周2 bug 修复"

- 日14 (2/15, 周日): 休息/优化 (加日志监控 WebSocket 流量)。
- 里程碑: 实时广播工作, 能看到在线用户, 但无冲突解决 (可能覆盖)。

周3: 冲突解决 + 版本历史 (2月16日-2月22日, 目标: 多人编辑不丢失内容)

- 日15 (2/16, 周一): 乐观锁实现
 - 用 @Version 字段: update 时 check version 匹配。
 - DocumentService 加 updateWithLock: findAndModify (带 version 查询)。
 - 如果冲突, 抛异常或合并。
 - commit: "feat: 乐观锁防覆盖"
- 日16 (2/17, 周二): 简单 OT / Patch 合并

- 用 diff-match-patch 库 (加依赖 com.googlecode.java-diff-utils:diffutils)。
- Service: applyPatch (计算 delta vs 当前 content, 合并后存)。
- 广播合并后 delta + 新 version。
- commit: "feat: 简单 OT patch 合并"

- 日17 (2/18, 周三): 版本历史存储

- DocumentEntity 的 versions List 加新 DocumentVersion (editorId, delta, timestamp)。
- 每次编辑 push 到 versions + save。
- 加 API: GET /documents/{id}/history (返回列表)。
- commit: "feat: 版本历史嵌入存储"

- 日18 (2/19, 周四): 前端冲突处理

- 前端收到冲突时: rebase 本地 delta 到最新 content。
- 加回滚按钮: 从 history 恢复版本。
- 测试: 3 人并发编辑, 看是否丢失。
- commit: "feat: 前端 OT 支持 + 历史 UI"

- 日19 (2/20, 周五): 权限 + 分享

- 加分享链接: 生成 inviteCode, 存 Redis (TTL 1天)。
- 编辑时 check owner 或 invited。
- commit: "feat: 文档分享权限"

- 日20 (2/21, 周六): 调试日

- 模拟高并发（用 JMeter 或多窗口），修复锁死/数据不一致。
- commit: "fix: 周3 bug 修复"
- 日21 (2/22, 周日): 休息/测试（邀请朋友试用，收集反馈）。
- 里程碑：冲突解决工作，版本历史可见，基本像简化 Google Docs。

周4：扩展 + 部署 + 文档（2月23日-2月29日，目标：上线 demo + portfolio 准备）

- 日22 (2/23, 周一): AI 扩展（OpenAI 摘要）
 - 加依赖 spring-boot-starter-webclient。
 - DocumentService 加 generateSummary：调用 OpenAI API (POST /chat/completions, prompt="总结文档")。
 - 前端按钮触发，显示摘要。
 - commit: "ext: 集成 OpenAI 摘要"
- 日23 (2/24, 周二): Google Docs API 模拟扩展
 - 模拟导入：加 API POST /import/google (用 WebClient 拉取模拟数据，存新文档)。
 - commit: "ext: 模拟 Google Docs 导入"
- 日24 (2/25, 周三): 优化 + 监控
 - 加 Spring Boot Actuator：依赖 actuator，暴露 /actuator/health。
 - Redis 限流：编辑频率用 Redis Incr 防刷。
 - commit: "opt: 性能监控 + 限流"
- 日25 (2/26, 周四): Docker 容器化

- 写 Dockerfile: FROM openjdk:21, COPY target/*.jar。
- docker-compose.yml: 后端 + Mongo + Redis。
- 测试本地 docker run。
- commit: "deploy: Docker 配置"
- 日26 (2/27, 周五): 云部署
 - 部署到 Render/Railway (免费), 配置环境变量 (JWT secret, Mongo URI)。
 - 加域名或分享链接。
 - commit: "deploy: 云上线"
- 日27 (2/28, 周六): 文档 + 演示
 - 写 README: 架构图 (用 Draw.io), 技术选型、挑战解决 (并发锁)、运行指南。
 - 录 demo 视频/GIF: 多人编辑过程。
 - commit: "docs: 项目 README + demo"
- 日28 (2/29, 周日): 最终调试 + 回顾
 - 整体测试, 修复遗留 bug。
 - 写博客/简历点: "实现了基于 OT 的实时协作, 处理了并发冲突"。
- 里程碑: 项目完整上线, 能在实习面试 demo 实时系统 + AI 扩展。

这个日程体现了你的能力: 从单体到实时分布式 (WebSocket + Redis), 解决并发问题。总计约 100-150 小时, 适合大二学生。完成后, 仓库会活跃, 简历加分巨大。如果时间紧, 周4 扩展可缩减。卡在哪天任务, 直接问我代码片段! 加油, 大帅, 新加坡时间还早, 2月起步正好💪